

АНОТАЦІЯ

Бакалаврська кваліфікаційна робота виконана студентом групи ІІІ-42 Ковалем Денисом Петровичем. Тема “Система для автоматичної нормалізації суржику та діалектних форм української мови”. Робота направлена на здобуття ступеня бакалавра за спеціальністю 122 “Комп’ютерні науки”.

Об’єктом дослідження є процеси автоматизованої обробки та нормалізації текстів природної мови українською мовою в умовах високої варіативності мовлення.

Предметом досліджень є методи побудови паралельних корпусів для низькоресурсних мовних варіантів та методи нейронного машинного перекладу/редагування і інструкційного донавчання великих мовних моделей для нормалізації діалектних і суржикових форм.

Досягнення мети відбувається за рахунок розробки baseline-моделі нормалізації для гуцульського діалекту та масштабування підходу на кілька діалектів і суржик із використанням інструкційного донавчання сучасної україномовної LLM (LaraLLM). Подальше опрацювання даних відбувається із використанням методів нейронного машинного перекладу та параметро-ефективних технік донавчання (LoRA/QLoRA).

У результаті виконання дипломної роботи створено систему автоматичної нормалізації діалектів і суржику до літературної української мови; розроблено програмну реалізацію з інтерфейсом користувача, що дозволяє обирати тип вхідного тексту (діалект/суржик) і отримувати нормалізований результат.

Загальний обсяг роботи: 46 сторінок, 5 рисунків, 18 посилань.

Ключові слова: нормалізація діалектів, суржик, машинне навчання, NLP, великі мовні моделі, LaraLLM.

ABSTRACT

The bachelor's qualification work was completed by a student of the ShI-42 group Koval Denys Petrovych. The topic is "System for automatic normalization of surzhyk and dialectal forms of the Ukrainian language". The work is aimed at obtaining a bachelor's degree in 122 "Computer Science".

The research addresses automated normalization of Ukrainian natural language texts, which exhibit high dialectal and stylistic variability.

The study examines methods for building parallel corpora for low-resource language varieties, neural machine translation and editing, and instruction fine-tuning of large language models for dialect and surzhyk normalization.

The work builds a baseline normalization model for the Hutsul dialect, then scales it to multiple dialects and surzhyk via instruction fine-tuning of LapaLLM, a Ukrainian-oriented LLM. The pipeline uses neural machine translation and parameter-efficient fine-tuning (LoRA/QLoRA).

The work produced a system that normalizes dialectal and surzhyk text to literary Ukrainian, including a user interface where the input type (dialect or surzhyk) can be selected and the normalized result retrieved.

The total volume of work: 46 pages, 5 figures, 18 references.

Keywords: dialect normalization, surzhyk, machine learning, NLP, large language models, LapaLLM.

ЗМІСТ

Зміст

АНОТАЦІЯ	1
ABSTRACT	2
ЗМІСТ	3
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	4
ВСТУП	5
1 ЗАГАЛЬНІ ПОЛОЖЕННЯ	7
1.1 Аналіз літературних джерел	7
1.2 Аналіз наборів даних	9
1.2.1 Опорні ресурси для гуцульського діалекту: ручний корпус, синтетика та словник	10
1.2.2 Джерело “норми”: використання Spivavtor як пулу літературної української	12
1.2.3 Генерація синтетичних даних: принципи, сценарії та контроль якості	13
1.2.4 Інтеграція джерел у єдиний корпус і організація даних у проєкті	14
1.2.5 Підготовка вибірок і балансування (для майбутнього мультидіалектного корпусу)	15
1.3 Висновки до розділу	16
2 ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ	17
2.1 Аналіз методів і підходів до нормалізації діалектів та суржику .	17
2.1.1 Нормалізація як задача перетворення тексту	17
2.1.2 Rule-based підходи та їх обмеження	18
2.1.3 Нейромережеві підходи: seq2seq і decoder-only LLM . . .	18
2.2 Постановка задачі та інформаційна модель даних	19
2.2.1 Формальна постановка	19

2.2.2	Подання даних для seq2seq	19
2.2.3	Подання даних для instruction-tuned LLM (chat format)	20
2.3	Математичні основи використаних моделей	20
2.3.1	Архітектура Transformer: увага та обчислення	20
2.3.2	Seq2seq encoder-decoder: умовна генерація	21
2.3.3	Decoder-only LLM: автогресивна модель	21
2.4	Вибір моделі та обґрунтування: від бейзлайну до Lora LLM	21
2.4.1	Бейзлайн для гуцульського діалекту як необхідний етап	21
2.4.2	Масштабування на мультидіалектність і суржик	21
2.4.3	Lora LLM: ключові властивості	22
2.4.4	Параметро-ефективне донавчання LLM: LoRA/QLoRA	23
2.5	Метрики якості та цільові функції навчання	23
2.5.1	Крос-ентропія та маскування втрат	23
2.5.2	Декодування: greedy та beam search	24
2.5.3	BLEU та chrF++ для нормалізації	25
2.5.4	Семантичні метрики та оцінювання якості	25
2.6	Інформаційне забезпечення навчання: баланс, семплінг, контроль доменів	25
2.6.1	Дисбаланс діалектів та принципи семплінгу	25
2.6.2	Мультизадачний формат і маркери режимів	25
2.7	Висновки до розділу	26
3	ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ	27
3.1	Засоби реалізації	27
3.2	Вимоги до технічного та програмного забезпечення	27
3.3	Висновки до розділу	27
	ЗАГАЛЬНІ ВИСНОВКИ	28
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	29
	Додаток А	31
	Додаток Б	32

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API - Application Programming Interface, програмний інтерфейс (інтерфейс програмування застосунків, інтерфейс прикладного програмування) - набір визначень підпрограм, протоколів взаємодії та засобів для створення програмного забезпечення.

AI - Artificial Intelligence - популярне англomовне скорочення для позначення “Штучного інтелекту”.

ML - Machine learning (Машинне навчання).

DL - Deep learning (Глибоке навчання).

UMAP - Uniform Manifold Approximation.

t-SNE - t-distributed stochastic neighbor embedding.

CART - Classification and Regression Tree.

ВСТУП

Актуальність теми зумовлена високою варіативністю української мови в реальній комунікації: поряд із літературною нормою поширені регіональні діалекти та російсько-український суржик. В Україні внутрішня міграція відбувалася й раніше, а після початку повномасштабного вторгнення суттєво зросла: люди опиняються в нових мовних середовищах, де діалектні слова, суржик або локальні форми можуть бути незрозумілими у побуті, навчанні чи роботі. Такі самі труднощі виникають в абітурієнтів і студентів із сіл, які переїжджають до міст. Тому автоматичний інструмент “діалект/суржик → літературна українська” корисний для нормалізації коротких текстів: повідомлень, оголошень, чатів.

Для автоматичної корекції української мови наявних інструментів небагато. Більшість орієнтована на стандартні орфографічні та граматичні помилки; семантична нормалізація діалектних або суржикових конструкцій підтримується рідко і здебільшого реалізується rule-based підходами. Трансформерні архітектури відкривають інший шлях: систему навчають безпосередньо на паралельних корпусах “нестандартний різновид → стандарт”, що дозволяє переносити підхід на різні мовні різновиди [1.]. Якість таких систем оцінюють класичними автоматичними метриками і сучасними нейронними фреймворками оцінювання якості перекладу [3.].

Метою бакалаврської кваліфікаційної роботи є розробка та експериментальне оцінювання системи автоматичної нормалізації українських діалектів і російсько-українського суржику до літературної української мови на основі нейронних моделей та підходів до роботи з низькоресурсними даними.

Задачі роботи:

- проаналізувати предметну область та сучасні підходи до нормалізації діалектів і гібридних мовних форм у задачах NLP;
- проаналізувати доступні джерела даних для задачі “діалект/суржик → літературна українська” та обґрунтувати спосіб формування навчального корпусу;
- розробити та експериментально оцінити baseline-рішення на прикладі гучульського діалекту, сформувавши відповідний паралельний корпус і навчивши модель нормалізації;

- масштабувати підхід на кілька діалектів та суржик, сформувавши багато-діалектний корпус і уніфікований пайплайн підготовки даних;
- провести інструкційне донавчання сучасної україномовної LLM (LaraLLM) для задачі нормалізації та виконати експериментальні дослідження якості на контрольних наборах [10.];
- реалізувати прикладну систему та продемонструвати її роботу.

Об'єктом дослідження є процеси автоматизованої обробки та нормалізації текстів природної мови українською мовою в умовах високої варіативності мовлення.

Предметом дослідження є методи побудови паралельних корпусів для низькоресурсних мовних варіантів, нейронного машинного перекладу та редагування, а також інструкційного донавчання великих мовних моделей для нормалізації діалектних і суржикових форм [1.], [10.].

Методи дослідження: аналіз і підготовка корпусів текстових даних, синтетична генерація паралельних прикладів на основі лінгвістичних правил і словникових ресурсів, навчання трансформерних моделей для побудови baseline, інструкційне донавчання великих мовних моделей із параметро-ефективними техніками, оцінювання якості за автоматичними метриками (BLEU, chrF++, TER) і підходами reference-free QE [3].

Результатом роботи є система нормалізації коротких україномовних текстів: спочатку побудовано і оцінено baseline для гуцульського діалекту, потім сформовано багатодіалектний корпус і спеціалізовано LaraLLM на задачу нормалізації. Для демонстрації реалізовано інтерфейс, що дозволяє обирати тип вхідного тексту і отримувати нормалізований результат. Систему можна застосовувати як самостійний інструмент або як компонент ширших україномовних NLP-пайплайнів.

1 ЗАГАЛЬНІ ПОЛОЖЕННЯ

1.1 Аналіз літературних джерел

Автоматична нормалізація діалектних форм і суржику до літературної норми поєднує підходи двох напрямів: нейронного машинного перекладу (Neural Machine Translation, NMT) та автоматичного виправлення тексту (Grammatical Error Correction, GEC). У контексті діалектів нормалізацію природно трактувати як “переклад” зі специфічного мовного різновиду на стандарт, тобто як NMT-задачу з низьким рівнем ресурсів. Подібні постановки активно досліджуються для інших мов: зокрема, у роботі [1.] показано, що для діалектного перекладу корисною може бути мультизадачність (Multitask Learning), яка допомагає моделі узагальнювати мовні закономірності за дефіциту даних. Для української мови діалектна нормалізація ускладнюється як варіативністю діалектів, так і наявністю суржику - гібридного змішаного мовлення.

Окремим викликом є те, що діалектні та суржикові тексти часто містять нестандартну орфографію, фонетичні відхилення, домішки іншої мови, варіативні морфологічні форми та неуніфіковані скорочення. У таких умовах переваги можуть мати моделі, менш чутливі до помилок токенізації. Наприклад, byte-рівневі підходи, як у ВуТ5, зменшують залежність від словникових сегментацій і потенційно краще працюють зі “шумними” та нестандартизованими входами [2.]. Для багатомовних сценаріїв вагомою є також стратегія збалансованого семплінгу під час переднавчання, яка впливає на якість для низькоресурсних мов; у цьому контексті важливі результати щодо Fair/Effective language sampling, наведені в [17.]. Такі підходи створюють теоретичне підґрунтя для вибору архітектур і базових моделей для задачі нормалізації.

Для коректного порівняння моделей та аналізу прогресу потрібне узгоджене оцінювання. У практиці NMT/GEC традиційно використовують BLEU, chrF++, TER та інші автоматичні метрики. Водночас для сучасних систем перекладу/редагування активно розвиваються нейронні методи оцінювання, зокрема COMET, який дозволяє оцінювати якість ближче до людських суджень і використовуватися як додатковий сигнал для аналізу якості [3.]. Також важливим є напрям quality estimation (оцінка якості без еталонного перекладу), який розглядає підходи від “ручних” ознак до великих мовних моделей [7.]. Для задачі діалектної нормалізації це актуально, оскільки один і той самий діалектний ви-

раз може мати кілька прийнятних нормалізованих варіантів.

З боку GEC і редагування текстів для української мови сформувались перші суттєві ресурси. UA-GEC - корпус для граматичної корекції та флюентності - перший систематичний ресурс для навчання моделей редагування українською [16.]. Іншим прикладом є інструкційно-донавчена модель Spivavtor, орієнтована на задачі україномовного редагування [14.]. Оскільки діалектна нормалізація частково перетинається з редагуванням (виправлення ненормативних форм, узгодження, стилістична нейтралізація), ці роботи створюють методологічну основу та надають корисні напрацювання щодо формату даних і способів навчання.

Основна проблема низькоресурсних задач - нестача розмічених пар “вхід → еталон”. У цьому контексті практики синтетичного розширення даних є стандартним інструментом. Робота [4.] (та її розширення [5.]) досліджує моделі, навчені на синтетичних даних для українського GEC, підтверджуючи, що синтетика може суттєво впливати на якість, якщо правильно сконструйовані правила “псування” (errorification) і контроль доменної близькості. Для діалектів і суржику аналогічна логіка застосовується при генерації паралельних пар: або через лінгвістичні правила та словники, або через LLM-керовану генерацію з додатковим контекстом.

Окремої уваги потребує суржик як явище мовного контакту. Лінгвістичні дослідження описують його як змішування українських і російських елементів, що проявляється не лише в лексиці, а й у морфосинтаксисі [9.]. З точки зору автоматичної обробки важливим є також виявлення/класифікація змішаного мовлення та його компонентів; цьому присвячені підходи до автоматичного розпізнавання змішаних мов (на прикладі суржику) [15.]. Ці джерела важливі для формалізації типових трансформацій “суржик → норма” та для вибору стратегій генерації даних.

З огляду на зростання ролі decoder-only LLM та інструкційного донавчання, актуальними є роботи про адаптацію сучасних архітектур до української. Зокрема, розглядаються практики донавчання Gemma та Mistral для покращення україномовної репрезентації [6.], а також з’являються україномовні або адаптовані під українську моделі, зокрема LaraLLM [10.] і MamaуLM [11.]. Для діалектної нормалізації інструкційний формат (“нормалізуй діалектний текст до літературної української”) є природним, оскільки дозволяє уніфікувати по-

становку для багатьох діалектів і суржику в межах однієї моделі.

Найбільш релевантним джерелом для побудови практичного рішення в контексті гуцульського діалекту є робота “Vuuko Mistral: Adapting LLMs for Low-Resource Dialectal Translation” [18.], де запропоновано підходи до формування паралельних даних і навчання моделей у низькоресурсному діалектному сценарії. Хоча у згаданій роботі напрям перекладу може відрізнятися від задачі нормалізації, запропоновані принципи побудови корпусу, словникова підтримка та підходи до масштабування є методологічно придатними й для постановки “діалект → стандарт”.

Аналіз показує: нормалізація діалектів і суржику методично близька до NMT і GEC; за браку даних визначальними є синтетичне розширення та інструкційне донавчання; для української існує базис із корпусів і моделей редагування, проте діалекти і суржик потребують спеціалізованих ресурсів; сучасні україномовні LLM дають природну основу для мультидіалектного підходу.

1.2 Аналіз наборів даних

Для задачі автоматичної нормалізації діалектних форм і російсько-українського суржику до літературної української мови визначальним фактором є якість і репрезентативність даних. На відміну від “класичного” машинного перекладу між мовами, у даній роботі маємо справу з мовними різновидами однієї мови (або змішаним мовленням), де межа між “помилкою”, “локальною нормою” та “варіантом написання” часто є розмитою. Через це вимоги до корпусу включають не лише достатній обсяг, але й наявність узгоджених еталонів, контроль джерел (ручні/синтетичні), баланс різних типів явищ (лексика, морфологія, синтаксис), а також прозору процедуру формування train/validation/test.

У межах роботи використано комбінований підхід: (1) як опорну базу взято реальні паралельні дані для гуцульського діалекту; (2) для масштабування покриття додано синтетичні пари, згенеровані за участі лінгвістичних правил і словникових ресурсів; (3) для формування “норми” літературної української та забезпечення різноманітності target-сторони використано україномовний датасет редагування/перекладу Spivavtor (використовується саме як джерело нормативних цільових форм та стилістично природних речень) [14.]. Далі наведено детальний аналіз кожного джерела даних та способу їх інтеграції.

1.2.1 Опорні ресурси для гуцульського діалекту: ручний корпус, синтетика та словник

Систему доцільно починати з діалекту, для якого є найбільше ресурсів. У роботі таким “якорем” обрано гуцульський діалект, оскільки для нього доступні (а) вручну анотований паралельний корпус, (б) синтетично згенерований корпус, (в) діалектний словник (рис. 1.1). Наявність цих компонентів дозволяє повністю відпрацювати процес: від формування датасету й розбиття на підвибірки до навчання моделі та аналізу помилок на реальних прикладах.

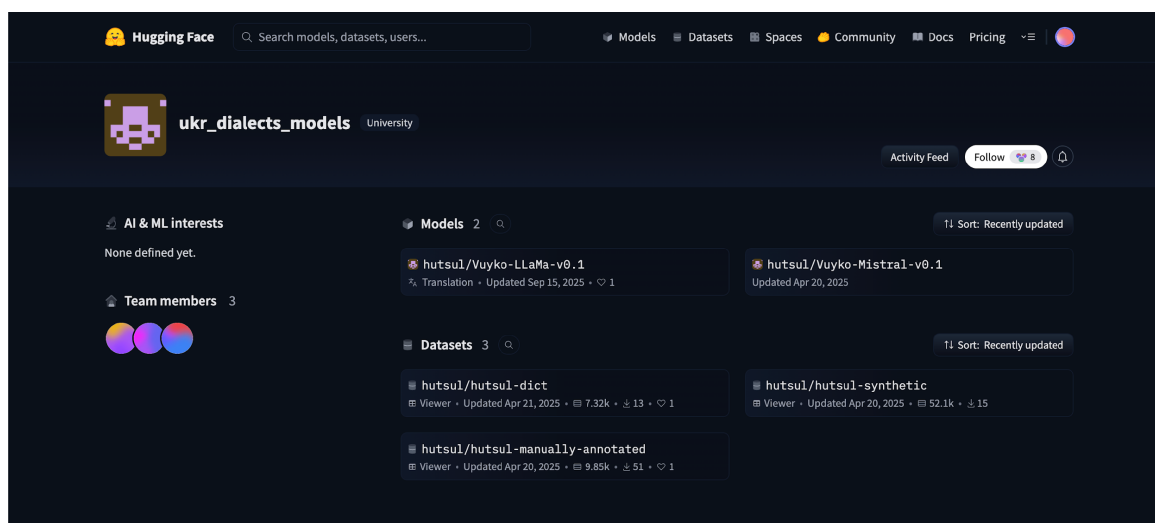


Рис. 1.1: Приклад сторінки з гуцульськими ресурсами на Hugging Face (датасети та моделі, що використовувались як вихідні джерела).

Ручний паралельний корпус. Вручну анотований корпус є найбільш цінним компонентом, оскільки забезпечує надійність відповідників “діалект → норма” (рис. 1.2). Саме ці дані рекомендовано використовувати для формування контрольних наборів (validation/test), щоб метрики відображали реальну якість на “справжніх” діалектних прикладах. Для такого корпусу стандартною є структура з двома колонками: source (діалектний текст) та target (літературний відповідник). У задачі нормалізації один діалектний вислів може мати кілька допустимих літературних варіантів; тому ручна розмітка, як правило, фіксує один узгоджений еталон, а альтернативи можуть розглядатись на етапі якісного аналізу.

Синтетичний паралельний корпус. Синтетичний корпус для гуцульського використовується як спосіб збільшення обсягу даних і покриття рідкісних слів/форм. Типова проблема для діалектів полягає в тому, що ручні корпуси часто є “малими” за обсягом, а їх домен може бути вузьким (наприклад, текс-

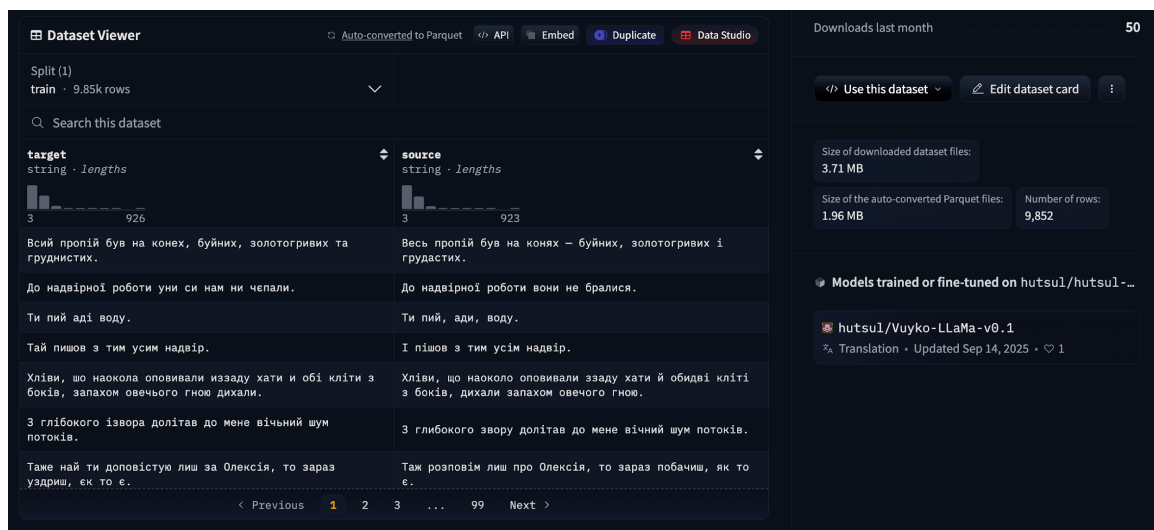


Рис. 1.2: Приклад перегляду вручну анованого корпусу для гуцульського діалекту (колонки *source/target* та статистика довжин прикладів).

ти певного жанру). Синтетичні пари дозволяють (а) розширити домен, (б) збільшити частоту специфічної лексики, (в) урізноманітнити структури речень. Разом із тим, синтетика не може замінити ручну розмітку для оцінювання, оскільки містить ризик “штучних” конструкцій і систематичних помилок генератора. Тому синтетичні дані доцільно використовувати переважно у *train*, а контрольні набори формувати із ручних прикладів.

Діалектний словник. Словник виконує кілька ролей:

- джерело лексичних відповідників (діалектна форма → літературна);
- “обмежувач” для генерації синтетики (підказує допустимі діалектні варіанти);
- retrieval-база (простий RAG): під час генерації можна підставляти у контекст приклади зі словника, щоб зменшити випадковість і підвищити узгодженість діалектних форм.

Словник також корисний для аналізу помилок: частину помилок моделі можна розкласти на (а) лексичні відповідники, (б) морфологічні/фонетичні трансформації, (в) синтаксичні перебудови. Якщо модель помиляється саме на словникових відповідниках - це ознака проблеми з покриттям або з частотністю відповідних пар у *train*.

1.2.2 Джерело “норми”: використання Spivavtor як пулу літературної української

Для задачі нормалізації важливо не лише “замінити діалектні слова”, а отримати природний і нормативний текст, що відповідає стилістичним очікуванням. Тому потрібне джерело літературної української з різноманітними конструкціями. У роботі для цього використовувався датасет Spivavtor (рис. 1.3), який містить пари редагування та перефразування українською мовою, зокрема підзадачі grammatical error correction (gec), paraphrase, simplification тощо [14].

У контексті даної роботи Spivavtor застосовувався не як “діалектний” корпус, а як:

1. пул цільових (target) речень, які вже є літературними та стилістично природними;
2. джерело різноманітних нормативних структур, корисних для синтетичної генерації пар;
3. орієнтир для стилю: цільова сторона нормалізації повинна бути не лише граматично правильною, але й “плинною”.

id	src	tgt	task
int64	string · lengths	string · lengths	string · classes
106.28k	990176	770153	gec
10%	34.3%	29.8%	44.5%
1	Перефразуйте: Кайс став одержимий нещодавно усвідомленою вразливістю.	Кайс був одержимий своєю нововиявленою вразливістю.	paraphrase
2	Перепишіть речення іншими словами: І вона більше не думатиме про Джонні Сміта та його...	Джонні Сміт і його дивна чарівна посмішка, про яку він ніколи більше не згадав.	paraphrase
3	Виправити граматику: З простими, там, поїсти, поспати не важко.	З простими, там, поїсти, поспати не важко.	gec
4	Виправте всі граматичні помилки: Тоді як ДНК має дві спіралі – інформаційний ряд...	Тоді як ДНК має дві спіралі – інформаційний ряд з'єднаний з комплементарним йому іншим...	gec
5	Зробіть цей текст легше для розуміння: Ви в дуже серйозній біді, містере Штайнфелд.	Містере Штайнфелд, у вас великі проблеми.	simplification
6	Виправте зв'язність в цьому тексті: англiканство вперше прийшло до Бразилії на...	Англiканство вперше прибуло до Бразилії на початку 19 століття, коли португальська...	coherence
7	Покращте зв'язність тексту: пісні у стилі блек-метал часто відрізняються від...	Блек-метал-пісні часто відрізняються від традиційної структури пісень і часто не...	coherence
8	Виправити граматику: Іван побачив сірий берет в гуші на початку Великої Микитської,...	Іван побачив сірий берет у гуші на початку Великої Микитської, але Герцена.	gec

Рис. 1.3: Приклад перегляду датасету Spivavtor (колонки src/tgt та мітка типу завдання).

Практична логіка використання Spivavtor як “норми” така: вибирається літературне речення y із Spivavtor (часто з підмножини, де текст є нейтральним за стилем і не надто довгим), після чого генерується діалектний або суржиковий варіант x - за правилами/словником/LLM. Таким чином формується нова пара (x, y) , де y гарантовано належить до літературної української, а x - штучно створений “нестандартний” варіант, який модель вчитиметься нормалізувати.

1.2.3 Генерація синтетичних даних: принципи, сценарії та контроль якості

Оскільки для більшості діалектів і суржику дефіцит ручних пар є критичним, синтетичне розширення є необхідним. В роботі застосовувався підхід “норма $\rightarrow$ нестандарт”, коли літературний текст виступає еталоном y , а діалектний/суржиковий варіант x генерується. Це має кілька переваг:

- легше отримати багато різноманітних літературних речень (зокрема з Spivavtor);
- діалектні правила/словники можна застосувати як контрольований механізм “відхилення” від норми;
- формування пар одразу гарантує наявність якісного target.

Умовно можна виділити три сценарії синтетики:

1. **Словниково-керована генерація (лексичні заміни + узгодження).** З літературного речення вибираються слова/фрази, для яких є діалектні відповідники у словнику, після чого виконуються заміни та мінімальні морфологічні корекції (закінчення, узгодження, частки). Такий підхід добре покриває лексичний шар, але слабший щодо синтаксичних перебудов.
2. **Правилова генерація (фонетика/морфологія/типові конструкції).** Застосовуються регулярні трансформації (наприклад, фонетичні варіанти написання, характерні суфікси, частки). Це підвищує “діалектність” форми, але потребує акуратності, щоб не зруйнувати читабельність або не створити нереалістичні сполучення.
3. **LLM-генерація з обмеженнями та RAG-підказками.** LLM генерує діалектний/суржиковий варіант, отримуючи як контекст: (а) інструкцію, (б) приклади діалектних відповідників зі словника, (в) (за потреби) невеликий набір демонстраційних пар з ручного корпусу.

су. Такий режим підвищує природність і різноманітність синтетики, але вимагає суворішого контролю якості.

Контроль якості синтетичних пар - критичний, оскільки “погана” синтетика здатна зіпсувати навчання. У роботі доцільно застосовувати такі евристики/фільтри:

- фільтрація дублікатів (однакові x або y);
- контроль довжин: надто короткі приклади неінформативні, надто довгі - часто містять помилки або жанровий шум;
- відсів пар з надто малою/надто великою різницею:
 - якщо x приблизно рівний y , приклад майже нічого не вчить моделі;
 - якщо відмінність надто велика і змінюється зміст, приклад може бути некоректним;
- словникова валідація: якщо в x присутні “діалектні” слова, перевіряти, чи вони існують у словнику/правилах;
- вибіркового ручний аудит: невелика частина синтетики перевіряється вручну для оцінки типових помилок генерації.

1.2.4 Інтеграція джерел у єдиний корпус і організація даних у проєкті

Після збору ручних і синтетичних даних для гуцульського діалекту та підготовки пулу літературних речень (Spivavtor target), корпус інтегрується у єдину структуру, придатну для навчання. Практично це означає:

- уніфікацію формату (source, target);
- додавання метаданих (діалект, тип джерела: manual/synthetic);
- об’єднання файлів у “merged” набір, який використовується як основний train-корпус.

Організація даних у репозиторії важлива з точки зору відтворюваності (рис. 1.4): по структурі директорій має бути зрозуміло, звідки походять дані, які файли є “сирими”, які - очищеними, і який файл є фінальним для навчання.

Окремо варто зафіксувати ключовий принцип: контрольні набори (validation/test) формуються з реальних (ручних) пар, тоді як синтетика використовується переважно в train. Це дозволяє уникнути переоцінки якості на “штучних” прикладах та отримувати метрики, близькі до реального застосування.

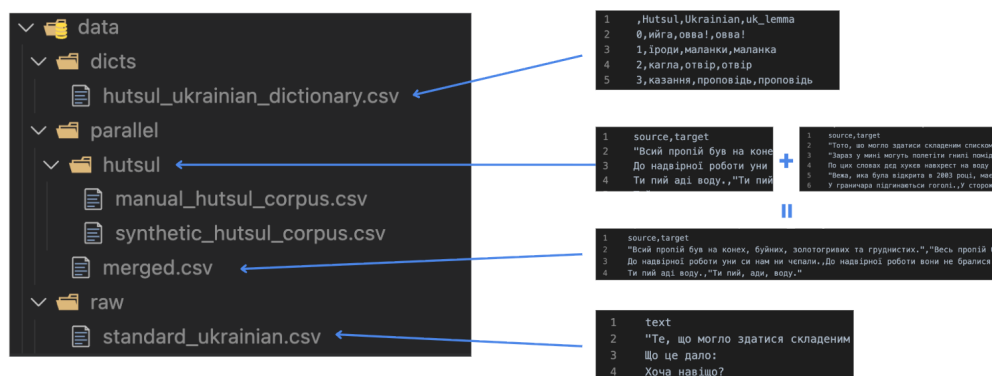


Рис. 1.4: Структура директорій даних у проєкті: словники, ручний корпус, синтетика, merged-корпус і пул літературної української.

1.2.5 Підготовка вибірок і балансування (для майбутнього мультидіалектного корпусу)

Навіть якщо базовий етап сфокусовано на одному діалекті, при переході до мультидіалектної нормалізації постає проблема дисбалансу: різні діалекти матимуть різний обсяг даних, різний домен (фольклор, побут, сучасні чати), різну відстань до літературної української. Якщо не враховувати це під час навчання, модель може:

- “переадаптуватись” до найбільшого діалекту;
- втратити якість на малих діалектах (catastrophic forgetting);
- генерувати “середній” стиль, який не є достатньо нормативним.

Тому вже на етапі формування даних доцільно закладати:

- стратифіковане розбиття за діалектами (і за типом джерела);
- балансування семплінгу під час навчання (ваги для діалектів);
- мінімальні квоти для рідкісних діалектів у train;
- окремі “діалект-специфічні” контрольні набори для метрик по кожному різновиду.

Ці принципи забезпечують масштабованість пайплайна: додавання нового діалекту стає процесом “додай дані + перезапусти уніфіковану підготовку”, без необхідності переписувати код або змінювати формат.

1.3 Висновки до розділу

У першому розділі проаналізовано літературні джерела та дані, необхідні для побудови системи автоматичної нормалізації діалектів і суржику до літературної української мови. Показано, що задача методично близька до NMT і текстового редагування та потребує нейромережевих підходів через високу варіативність діалектних і суржикових форм. Основною практичною складністю є низькоресурсність: для більшості діалектів відсутні достатні паралельні корпуси, тому ключовими стають синтетичне розширення даних і використання зовнішніх ресурсів.

Обґрунтовано вибір гуцульського діалекту як стартового: наявність вручну анотованого корпусу та діалектного словника дозволяє побудувати контрольний baseline-пайплайн і отримати надійну оцінку якості на реальних прикладах. Для формування “норми” літературної української використано датасети редагування (зокрема Spivavtor), що підвищує природність цільових речень і дає опору для генерації синтетики [14.]. Визначені принципи даних і підготовки корпусу формують основу для подальшого опису методів навчання, оцінювання та реалізації системи в наступних розділах.

2 ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

Розділ містить формалізацію задачі нормалізації, опис двох підходів (seq2seq та decoder-only LLM), обґрунтування вибору архітектур, а також математичні основи навчання, декодування та оцінювання. Узагальнену схему системи показано на рис. 2.1.

4-Phase ML Pipeline:

Phase 1: Data Generation
Input: Standard Ukrainian text
Tool: OpenRouter/Local
Output: Parallel corpus (CSV: source → target)

Phase 2: Model Training
Input: Parallel corpus (data/parallel/merged.csv)
Tool: PyTorch • Transformers
Model: Encoder-Decoder (umT5) or Decoder-Only (Llama/Gemma/Lapa/Mamay)
Output: Fine-tuned checkpoint (models/)

Phase 3: Evaluation
Input: Test set (data/parallel/eval.csv)
Tool: PyTorch • Transformers
Metric: BLEU/chrF++/TER
Output: predictions.txt + references.txt

Phase 4: Inference
Input: Dialect/surzhyk text
Tool: PyTorch • Transformers • Gradio
Output: Normalized Ukrainian text

Рис. 2.1: Узагальнена схема системи: (а) бейзлайн “гуцульський → літературна”, (б) мультидіалектний/суржиковий етап на Lapa LLM, (в) демо-інтерфейс.

2.1 Аналіз методів і підходів до нормалізації діалектів та суржику

2.1.1 Нормалізація як задача перетворення тексту

Нормалізацію діалектних форм та суржику можна розглядати як кероване перетворення тексту, де необхідно зберегти зміст, але змінити форму (лексичну, морфологічну, орфографічну, інколи синтаксичну) відповідно до літературної норми. На відміну від “виправлення граматики” у вузькому сенсі, тут часто присутні лексичні заміни (діалектизми), варіативна орфографія, фонетично мотивовані написання, а також код-міксинг (для суржику).

Формально будемо мати вхідний рядок x (діалект/суржик) та бажаний вихідний рядок y (літературна українська). Мета - побудувати модель f_θ , що апроксимує відображення:

$$f_\theta : x \mapsto y$$

З погляду статистичного моделювання це еквівалентно оцінюванню умовного розподілу $p_\theta(y|x)$.

Практичні особливості задачі:

- часткова тотожність: значна частина токенів у x може збігатися з y ;
- асиметрія трансформацій: “виправляти занадто багато” небезпечно (ри-

зик спотворити зміст), але “виправляти занадто мало” - не виконати нормалізацію;

- багатоваріантність норми: одна діалектна форма може мати декілька літературних парафразів;
- низькоресурсність: для більшості діалектів немає великих вручну розмічених паралельних корпусів.

2.1.2 Rule-based підходи та їх обмеження

Rule-based нормалізація опирається на словники відповідників, регулярні правила перетворень і евристики. Її перевага - контрольованість і пояснюваність: якщо відомі правила заміни, можна гарантувати певну поведінку. Однак для українських діалектів і суржику rule-based підхід зазвичай має такі проблеми:

- складно охопити всю лексичну варіативність;
- важко правильно моделювати контекстні значення (омонімія/полісемія);
- правила погано працюють із довгими залежностями і перебудовами;
- якість деградує при зміні домену (побутові чати, оголошення, студентські конспекти тощо).

Тому rule-based ресурси (словники/правила) у цій роботі розглядаються як допоміжні: (а) для побудови синтетичних даних; (б) для підказок (retrieval) під час генерації синтетики; (в) як інструмент аналізу помилок.

2.1.3 Неймережеві підходи: seq2seq і decoder-only LLM

У роботі послідовно застосовано дві парадигми.

1. Seq2seq (encoder-decoder) для бейзлайну на одному діалекті (гуцульському).

Переваги:

- стабільне навчання на паралельних даних;
- природна постановка як “переклад” $x \rightarrow y$;
- добре підходить для невеликих/середніх моделей і обмежених ресурсів.

2. Decoder-only LLM (instruction-tuned) для розширення на багато діалектів і суржик.

Переваги:

- універсальність: одна модель може виконувати різні підзадачі (нормалі-

зація різних діалектів, суржик, пояснення правок);

- простіший формат даних через інструкції (prompt → response);
- потенційно краща генеративна “плинність” та стилістична якість.

Ключовою базовою моделлю для масштабування обрано Lora LLM, оскільки це відкрита українсько-орієнтована LLM, побудована на базі Gemma-3-12B та оптимізована під українську за рахунок адаптації токенизатора і прозорого пайплайна тренування [10.].

2.2 Постановка задачі та інформаційна модель даних

2.2.1 Формальна постановка

Нехай Σ - алфавіт символів (або множина байтів/токенів), тоді вхідний текст:

$$x = (x_1, x_2, \dots, x_{|x|}), \quad x_i \in \Sigma$$

вихідний текст:

$$y = (y_1, y_2, \dots, y_{|y|}), \quad y_j \in \Sigma$$

Задача - знайти параметри θ , що максимізують правдоподібність тренувальних пар $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$:

$$\theta^* = \arg \max_{\theta} \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)})$$

У практичній реалізації використовується мінімізація негативного лог-правдоподібності (крос-ентропії):

$$L(\theta) = - \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)})$$

2.2.2 Подання даних для seq2seq

Для encoder-decoder моделі дані зберігаються у вигляді пар:

- source: діалект/суржик;
- target: літературна українська.

Після токенизації отримуємо послідовності індексів $x = (x_1, \dots, x_T)$, $y = (y_1, \dots, y_{T'})$. Модель навчається прогнозувати y_t з урахуванням усіх x та попередніх $y_{<t}$:

$$p_\theta(y|x) = \prod_{t=1}^{T'} p_\theta(y_t|x, y_{<t})$$

2.2.3 Подання даних для *instruction-tuned LLM (chat format)*

Для decoder-only LLM зручно представляти задачу як інструкцію, де модель отримує:

- роль “system” із правилами нормалізації;
- роль “user” із текстом (і, за потреби, міткою діалекту/режиму);
- роль “assistant” із відповіддю-нормою (рис. 2.2).

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."
* Assistant: "...нормалізований текст..."
```

Рис. 2.2: Приклад шаблону інструкції для LLM: system/user/assistant та місце для маркера діалекту.

2.3 Математичні основи використаних моделей

2.3.1 Архітектура Transformer: увага та обчислення

Основою і для seq2seq, і для decoder-only підходів є Transformer, який використовує механізм самоуваги. Нехай на вході є матриця представлень токенів $H \in \mathbb{R}^{T \times d}$. Для одного attention-блоку обчислюються:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V$$

де $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$.

Далі:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Multi-head attention використовує кілька голів і конкатенацію результатів:

$$\text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

2.3.2 Seq2seq encoder-decoder: умовна генерація

Encoder перетворює вхід x у контекстні представлення E . Decoder генерує y , використовуючи:

- self-attention по вже згенерованих токенах,
- cross-attention до E .

2.3.3 Decoder-only LLM: автогресивна модель

Decoder-only LLM моделює послідовність tokenів z (де z включає system+user+assistant у вигляді одного “ланцюжка”):

$$p_{\theta}(z) = \prod_{t=1}^{|z|} p_{\theta}(z_t | z_{<t})$$

На практиці нас цікавить якість саме на токенах відповіді assistant, тому під час навчання можна маскувати loss так, щоб оптимізувати лише частину “assistant”.

2.4 Вибір моделі та обґрунтування: від бейзлайну до Lora LLM

2.4.1 Бейзлайн для гуцульського діалекту як необхідний етап

Першим етапом реалізовано бейзлайн для гуцульського діалекту - з трьох причин:

1. дозволяє перевірити, що паралельні дані, токенизація та навчання працюють коректно;
2. дає контрольований “полігон” для аналізу помилок;
3. створює поріг якості, від якого можна вимірювати виграш при переході до LLM (рис. 2.3).

Результати оцінювання якості бейзлайну на валідаційній вибірці показано на рис. 2.4. Метрика BLEU стабілізується на рівні 51, що свідчить про адекватність моделі для задачі нормалізації.

2.4.2 Масштабування на мультідіалектність і суржик

Перехід від одного діалекту до множини діалектів та суржику ускладнює задачу:

- зростає різноманітність фонетичних і лексичних явищ;
- з’являється потреба у “перемикачі” режимів (який діалект нормалізуємо);

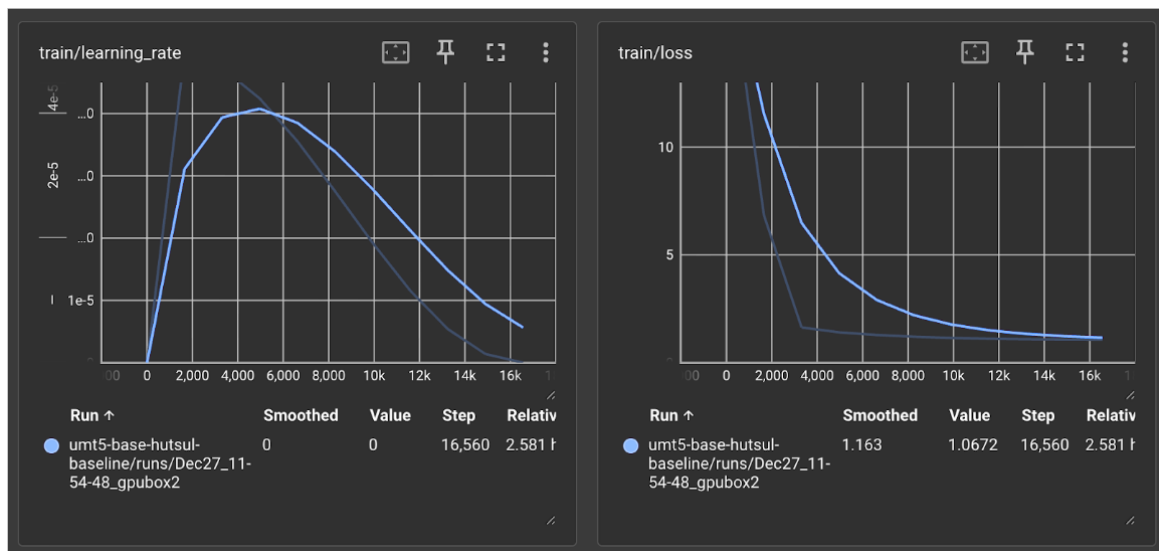


Рис. 2.3: Графік навчання бейзлайну (*learning rate та loss*).

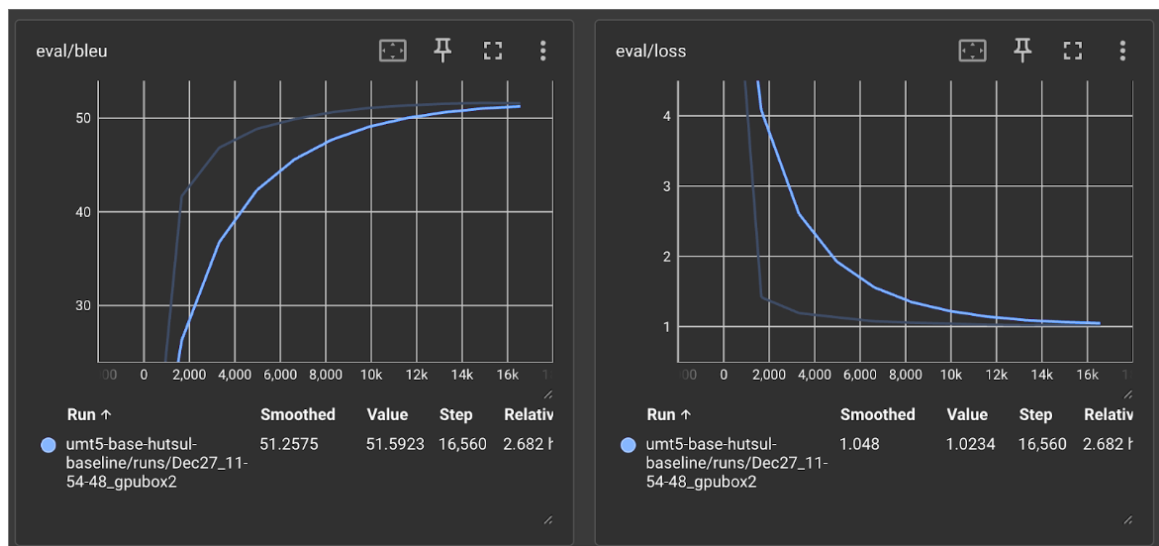


Рис. 2.4: Результати оцінювання бейзлайну: метрика BLEU (ліворуч) та validation loss (праворуч) під час навчання.

- необхідна краща генеративна здатність для природного стилю.

Decoder-only LLM з інструкційним форматом природно підтримує цю постановку: можна додавати в prompt маркер діалекту або навіть просити модель самостійно визначити різновид і нормалізувати.

2.4.3 Lora LLM: ключові властивості

Для мультидіалектного етапу використано Lora LLM [10.] (рис. 2.5) - відкриту українсько-орієнтовану модель, що базується на Gemma-3-12B.

Властивості, які прямо впливають на якість і ефективність нормалізації:

1. **Український токенізатор** - замінено приблизно 80 тис. токенів із

250 тис. на українські, що дає скорочення кількості потрібних токенів приблизно у 1.5 раза.

2. **Instruction-tuned версія** - спрощує інтеграцію в прикладний сервіс.
3. **Довгий контекст** - підтримка до 128K токенів відкриває шлях до RAG-підходів.
4. **Відкритість та відтворюваність** - публікується код і набори даних.

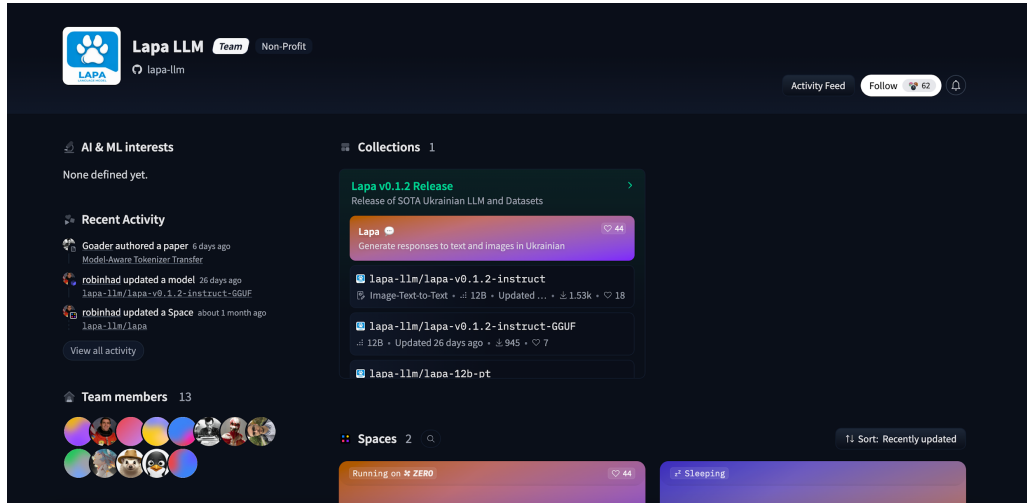


Рис. 2.5: Сторінка моделі Lapa LLM на Hugging Face.

2.4.4 Параметро-ефективне донавчання LLM: LoRA/QLoRA

Повне донавчання 12B-моделі є дорогим, тому практично застосовується PEFT (parameter-efficient fine-tuning), найчастіше LoRA (рис. 2.6).

Нехай у Transformer є матриця ваг $W \in \mathbb{R}^{d \times k}$. LoRA додає низькорангову поправку:

$$W' = W + \Delta W, \quad \Delta W = BA$$

де $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, а $r \ll \min(d, k)$.

Тоді кількість тренуваних параметрів зменшується з dk до $r(d + k)$.

2.5 Метрики якості та цільові функції навчання

2.5.1 Крос-ентронія та маскування втрат

Стандартна функція втрат для генеративних моделей:

$$L = - \sum_{t=1}^{T'} \log p_{\theta}(y_t | x, y_{<t})$$

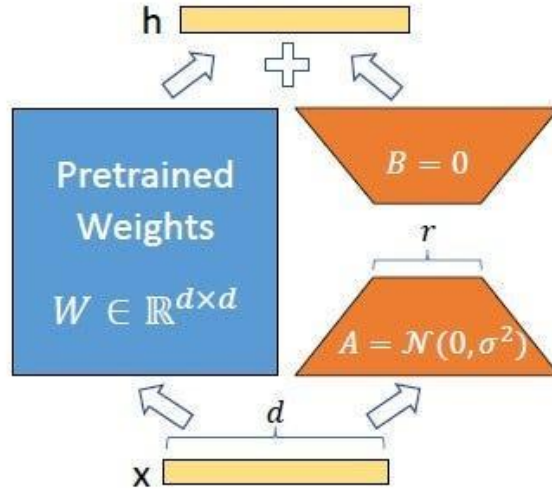


Рис. 2.6: Ілюстрація ідеї LoRA: замість повного оновлення ваг - низькорангова поправка в attention/FFN шарах.

Для instruction-tuned LLM доцільно оптимізувати лише токени відповіді assistant. Нехай $m_t \in \{0, 1\}$ - маска (1 лише на позиціях відповіді), тоді:

$$L_{masked} = - \sum_{t=1}^T m_t \log p_{\theta}(z_t | z_{<t})$$

2.5.2 Декодування: greedy та beam search

Під час інференсу потрібно знайти:

$$\hat{y} = \arg \max_y \log p_{\theta}(y|x)$$

Greedy-декодування:

$$\hat{y}_t = \arg \max_{y_t} p_{\theta}(y_t | x, \hat{y}_{<t})$$

Beam search підтримує B найкращих часткових гіпотез. Часто застосовують штраф за довжину (length penalty).

2.5.3 BLEU та chrF++ для нормалізації

Оскільки задача близька до перекладу, застосовні метрики MT. BLEU базується на модифікованій n-грамній точності та штрафі за короткість:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

де p_n - модифікована точність n-грам, w_n - ваги, а BP (brevity penalty) штрафує за короткі гіпотези.

Для мов із багатою морфологією символні метрики часто корисніші. chrF/chrF++ вимірює F-міру на символних n-грамах.

2.5.4 Семантичні метрики та оцінювання якості

Для нормалізації важливо не лише буквальне співпадіння, а і збереження змісту. Тому доцільно застосовувати нейромережеві метрики, наприклад COMET [3.], які корелюють з людськими оцінками якості перекладу/редагування.

2.6 Інформаційне забезпечення навчання: баланс, семплінг, контроль доменів

2.6.1 Дисбаланс діалектів та принципи семплінгу

У мультидіалектному корпусі обсяги різних діалектів майже неминуче різні. Якщо навчати модель на “сирому” об’єднанні, то оптимізація зміститься до найбільшого діалекту. Один із способів - вводити ваги семплінгу:

$$P(\text{dialect} = i) \propto n_i^\gamma$$

де n_i - кількість прикладів для діалекту i , а $\gamma \in [0, 1]$ керує вирівнюванням.

2.6.2 Мультитасковий формат і маркери режимів

Для LLM зручно навчати один “універсальний” формат:

- нормалізація діалекту (із маркером діалекту),
- нормалізація суржику (із маркером “SURZHUK”),
- за потреби - варіант “нормалізуй і коротко поясни правки” (окремий режим).

Тоді система фактично розв’язує множину умовних задач $p_\theta(y|x, t)$, де t -

тип/режим.

2.7 Висновки до розділу

У розділі наведено математичну постановку задачі нормалізації як умовної генерації $p_{\theta}(y|x)$, описано два послідовні етапи підходу: бейзлайн encoder-decoder для одного діалекту та масштабування через instruction-tuned decoder-only LLM. Подано ключові формули Transformer-уваги, цільових функцій (крос-ентропія, маскування для assistant), принципів декодування (greedy/beam search) та метрик оцінювання (BLEU, chrF++, нейромережеві метрики). Окремо обґрунтовано вибір Lora LLM як українсько-орієнтованої бази на Gemma-3-12B із адаптованим токенизатором, довгим контекстом та відкритим пайплайном [10.].

3 ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ

3.1 Засоби реалізації

Систему реалізовано на Python. Оскільки алгоритми машинного навчання і нейронних мереж не входять до стандартної бібліотеки мови, використано спеціалізовані пакети:

- **PyTorch** - основний фреймворк для побудови та навчання нейронних мереж;
- **Hugging Face Transformers** - бібліотека для роботи з трансформерними моделями, токенизаторами та пайплайнами;
- **PEFT (Parameter-Efficient Fine-Tuning)** - бібліотека для LoRA/QLoRA донавчання;
- **datasets** - бібліотека для роботи з наборами даних;
- **evaluate** - бібліотека для обчислення метрик якості (BLEU, chrF++ тощо);
- **Gradio** - фреймворк для створення демо-інтерфейсу.

3.2 Вимоги до технічного та програмного забезпечення

Для виконання поставленого завдання використовувався Python без хмарного середовища. При розробці бейзлайн-моделі тренування відбувалось на CPU, що є достатнім для невеликих seq2seq моделей. Для донавчання великих мовних моделей (Lara LLM на базі Gemma-3-12B) необхідне GPU з достатнім обсягом відеопам'яті (рекомендовано ≥ 24 GB для QLoRA).

Мінімальні вимоги до системи:

- Python 3.10+;
- RAM: ≥ 16 GB;
- Для бейзлайну: CPU достатньо;
- Для LLM: NVIDIA GPU з CUDA (A100/H100 або RTX серії для QLoRA).

3.3 Висновки до розділу

Розглянуто засоби реалізації: Python з бібліотеками PyTorch, Hugging Face Transformers, PEFT та Gradio. Описано вимоги до обчислювальних ресурсів для бейзлайну та LLM-етапу.

ЗАГАЛЬНІ ВИСНОВКИ

У роботі розроблено систему автоматичної нормалізації суржику та діалектних форм до літературної української мови.

Основні результати роботи:

1. Проаналізовано предметну область та виконано огляд сучасних підходів до нормалізації діалектів і гібридних мовних форм у задачах NLP.
2. Проаналізовано доступні джерела даних для задачі “діалект/суржик → літературна українська” та обґрунтовано спосіб формування навчального корпусу.
3. Розроблено та експериментально оцінено baseline-рішення на прикладі гуцульського діалекту.
4. На основі отриманих результатів масштабовано підхід на кілька діалектів та суржик.
5. Виконано інструкційне донавчання сучасної україномовної LLM (LaraLLM) для задачі нормалізації.
6. Реалізовано прикладну систему з інтерфейсом користувача для демонстрації роботи.

Система придатна для приведення коротких текстів до літературної норми і може слугувати окремим інструментом або компонентом більших україномовних NLP-пайплайнів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. L. H. Baniata, S. Park, i S.-B. Park, “A Neural Machine Translation Model for Arabic Dialects That Utilizes Multitask Learning (MTL)”, *Computational Intelligence and Neuroscience*, 2018, с. 1–10, Груд 2018, doi: 10.1155/2018/7534712.
2. L. Xue et al., “ByT5: Towards a token-free future with pre-trained byte-to-byte models”, Березень 2022, arXiv:2105.13626. doi: 10.48550/arXiv.2105.13626.
3. R. Rei, C. Stewart, A. C. Farinha, i A. Lavie, “COMET: A Neural Framework for MT Evaluation”, Жовтень 2020, arXiv:2009.09025. doi: 10.48550/arXiv.2009.09025.
4. M. Bondarenko, A. Yushko, A. Shportko, i A. Fedorych, “Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction”.
5. M. Bondarenko, A. Yushko, A. Shportko, i A. Fedorych, “Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction”.
6. A. Kiulian et al., “From Bytes to Borsch: Fine-Tuning Gemma and Mistral for the Ukrainian Language Representation”, Квітень 2024, arXiv:2404.09138. doi: 10.48550/arXiv.2404.09138.
7. H. Zhao et al., “From Handcrafted Features to LLMs: A Brief Survey for Machine Translation Quality Estimation”, *IJCNN 2024*, Чер 2024, с. 1–10. doi: 10.1109/IJCNN60899.2024.10650457.
8. R. Kovalchuk, M. Romanyshyn, i P. Ivaniuk, “Introducing OmniGEC: A Silver Multilingual Dataset for Grammatical Error Correction”.
9. K. Kent, “Language Contact: Morphosyntactic Analysis of Surzhyk Spoken in Central Ukraine”, *Language Contact*.
10. “LapaLLM”.
Доступно у: huggingface.co/lapa-llm
11. “MamayLM”.
Доступно у: huggingface.co/blog/INSAIT-Institute/mamaylm
12. R. Fedchuk i V. Vysotska, “Mathematical model of a decision support system

for identification and correction of errors in Ukrainian texts based on machine learning”.

13. M. Haliuk, “On the Path to Make Ukrainian a High-Resource Language”.
14. A. Saini, A. Chernodub, V. Raheja, i V. Kulkarni, “Spivavtor: An Instruction Tuned Ukrainian Text Editing Model”, Квітень 2024, arXiv:2404.18880. doi: 10.48550/arXiv.2404.18880.
15. N. Sira, G. M. D. Nunzio, i V. Nosilia, “Towards an Automatic Recognition of Mixed Languages in R: The Case of Ukrainian-Russian Hybrid Language Surzhyk”.
16. O. Syvokon, O. Nahorna, P. Kuchmiichuk, i N. Osidach, “UA-GEC: Grammatical Error Correction and Fluency Corpus for the Ukrainian Language”, *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, 2023, с. 96–102. doi: 10.18653/v1/2023.unlp-1.12.
17. H. W. Chung et al., “UniMax: Fairer and more Effective Language Sampling for Large-Scale Multilingual Pretraining”, Квітень 2023, arXiv:2304.09151. doi: 10.48550/arXiv.2304.09151.
18. R. Kyslyi, Y. Maksymiuk, i I. Pysmennyi, “Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation”, Червень 2025, arXiv:2506.07617. doi: 10.48550/arXiv.2506.07617.

Додаток А

Додаток Б